assignment no:6

6. Use Docker Compose to manage multiple containers for a web-based AI app, including containers for the AI model, database, and frontend.

In [ ]:

```
### vs code
ai-docker-app
│
├── docker-compose.yml
│
├── model
│  ├── app.py
│  ├── requirements.txt
│  └── Dockerfile
│
└── frontend
├── index.html
└── Dockerfile

3️⃣ Add Backend Code
 nano app.py
Paste:
from flask import Flask
from pymongo import MongoClient

app = Flask(__name__)

client = MongoClient("mongodb://mongodb:27017/")
db = client.testdb

@app.route("/")
def home():
    return "AI Model Running"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

4️⃣ Add requirements.txt
##  requirements.txt

flask
pymongo

5️⃣ Create Model Dockerfile
## Open model → Dockerfile
FROM python:3.9
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
CMD ["python", "app.py"]
```

```
6️⃣ Create Frontend
##Open frontend → index.html
<!DOCTYPE html>
<html>
  <head>
    <title>AI Docker App</title>
  </head>
  <body>
    <h1>AI Docker Application</h1>
    <p>Frontend running in Nginx container.</p>
  </body>
</html>

7️⃣ Create Frontend Dockerfile
## Open frontend → Dockerfile
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

8️⃣ Create docker-compose.yml
## In the main folder create docker-compose.yml
Paste:
services:
  mongodb:
    image: mongo
    container_name: mongo_db
    ports:
      - "27018:27017"

  model:
    build: ./model
    ports:
      - "5001:5000"
    depends_on:
      - mongodb

  frontend:
    build: ./frontend
    ports:
      - "8081:80"
    depends_on:
      - model

9️⃣ Open Terminal in VS Code
Click:
Terminal → New Terminal
Make sure you are inside:
ai-docker-app
Check using:
## ls
You should see:
docker-compose.yml
frontend
model

🔟 Build and Run Containers
```

```
Run this command:
## docker compose up --build

1️⃣1️⃣ Check Running Containers
Open new terminal in VS Code:
## docker ps
Expected:
mongo_db
ai-docker-app-model-1
ai-docker-app-frontend-1

1️⃣2️⃣ Test Frontend
Open browser:
## http://localhost:8081
You should see:
AI Docker Application
Frontend running in Nginx container.

1️⃣3️⃣ Test
Backend
In VS Code
terminal:
## curl

## http://localhost:5001
Output:
AI Model Running

1️⃣4️⃣ Test MongoDB
Run:
## docker exec -it mongo_db mongosh
Then:
## show dbs
## use testdb
## db.users.insertOne({name:"testuser"})
## db.users.find()
Exit:
## exit

1️⃣5️⃣ Stop Containers
Press:
CTRL + C
or run:
## docker compose down

10. Check Container Logs
Frontend logs:
## sudo docker logs ai-docker-app_frontend_1
Model logs:
## sudo docker logs ai-docker-app_model_1
MongoDB logs:
## sudo docker logs mongo_db
```

assignment 1 & 2

In [ ]:

```
##  sudo apt update
##  sudo apt install git
## git config –global user.name "usename"
## git config –global user.email "email"
## create directory in terminal : mkdir “your directory name”
##  cd directoryname/
## git clone "url"
## ls
## cd “your repo name”/
## ls
## cat README.md
## nano add.py
15. gedit add.py
16. git status
17. git add add.py
18. ls
19. git status
20. git commit -m "Added add .py file"
21. git status
22.git remote add origin ( URL)
22. git push origin main (token)

 After 2
# git checkout -b feature-login
# nano index.html
def login():
    username = input("Enter username: ")
    password = input("Enter password: ")

    if username == "admin" and password == "1234":
        print("Login successful!")
    else:
        print("Invalid credentials")

login()

#git add index.py

git commit -m "Added login page"
git push origin feature-login

git checkout -b feature-dashboard
nano dashbord.py
def dashboard():
    print("Welcome to the Dashboard!")
    print("1. View Reports")
    print("2. Manage Users")

dashboard()
git add dashboard.py
git commit -m "Added dashboard page"
git push origin feature-dashboard

git checkout main
git pull origin main
```

```
git merge feature-login
git merge feature-dashboard
# Resolve conflicts if any
git push origin main
```

assignment no:8

Implement auto-scaling for your AI app on Kubernetes and monitor its health and performance metrics.

In [ ]:
```
Install Required Tools
Install Docker:
## sudo apt update
## sudo apt install docker.io -y
Start Docker:
## sudo systemctl start docker
## sudo systemctl enable docker
Check Docker:
## docker --version

Install Kubernetes CLI:
## sudo snap install kubectl –classic
Check:
## kubectl version --client

Install Minikube:

## curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-

## sudo install minikube-linux-amd64 /usr/local/bin/minikube
Check:
## minikube version

Start Kubernetes Cluster
Start Minikube:
## minikube start –driver=docker
Verify cluster:
## kubectl get nodes
Output should show:

 minikube  Ready

Create Project Folder
## mkdir ai-k8s-app
## cd ai-k8s-app

Create AI Application
Create file:
## nano app.py
Paste:
from flask import Flask, request, jsonify

app = Flask(__name__)
```

```
@app.route("/")
def home():
    return "AI Model Running in Kubernetes"

@app.route("/predict", methods=["POST"])
def predict():
    data = request.json["number"]
    result = data * 2
    return jsonify({"result": result})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)


Create requirements file:
## nano requirements.txt
Flask

Create Dockerfile
## nano Dockerfile
Paste:
FROM python:3.9

WORKDIR /app

COPY . /app

RUN pip install -r requirements.txt

CMD ["python", "app.py"]


Build Docker Image
## sudo docker build -t ai-model .
Check image:
## sudo docker images

Load Image into Minikube
## minikube image load ai-model

Create Kubernetes Deployment
Create file:
## nano deployment.yaml
Paste:
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ai-app
  template:
    metadata:
      labels:
```

```
        app: ai-app
    spec:
      containers:
        - name: ai-container
          image: ai-model
          imagePullPolicy: Never
          ports:
            - containerPort: 5000
          resources:
            requests:
              cpu: "100m"
            limits:
              cpu: "500m"



Create Kubernetes Service
Create file:
## nano service.yaml
Paste:
apiVersion: v1
kind: Service
metadata:
  name: ai-service
spec:
  type: NodePort
  selector:
    app: ai-app
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30007
Deploy service:
## kubectl apply -f service.yaml
Check:
## kubectl get svc

Access Application
Run:
## minikube service ai-service
Browser opens showing:
AI Model Running in Kubernetes

********if this command is fail*****
    # kubectl apply -f deployment.yaml
    # kubectl get pods
    after run
    # minikube service ai-service

1️⃣1️⃣ Enable Metrics Server
Auto-scaling requires metrics.
## minikube addons enable metrics-server
Verify:
kubectl top pods
Example output:
ai-app-xxxxx 30m 100Mi
```

```
1️⃣2️⃣ Create Horizontal Pod Autoscaler
## kubectl autoscale deployment ai-app --cpu-percent=50 --min=2 --max=10
********if not run above then run *****
##kubectl autoscale deployment ai-app --cpu=50% --min=2 --max=10
*******
Check autoscaler:
## kubectl get hpa
Example:
ai-app cpu: 10%/50% min 2 max 10

1️⃣3️⃣ Generate Load
Run:
## while true; do curl $(minikube service ai-service --url); done
**** if not run above
✅ Option 1 (Best - Simple)

First get URL:

minikube service ai-service --url

👉 Output:

http://127.0.0.1:36289
******
1️⃣4️⃣ Observe Pod Scaling
In another terminal:
## kubectl get pods -w
Pods will increase:
ai-app-1
ai-app-2
ai-app-3
ai-app-4
This proves auto-scaling works.

1️⃣5️⃣ Monitor Health and Performance
Check CPU usage:
## kubectl top pods
Check nodes:
## kubectl top nodes
Check pod health:
## kubectl describe pod <pod-name>
Check logs:
## kubectl logs <pod-name>

1️⃣6️⃣ Final Commands (for Assignment
Output)
Run these:
## kubectl get pods
## kubectl get svc
## kubectl get hpa
## kubectl top pods
## kubectl describe hpa ai-app
These show:
AI app deployed
Service exposed
```

```
Metrics monitoring
Auto-scaling implemented
```

ASSIGNMENT NO : 7

7. Set up a local Kubernetes cluster with Minikube to deploy and scale your Dockerized AI application.

In [ ]:

```
1️⃣ Install Docker
Step 1
Update system packages.
## sudo apt update

Step 2
Install Docker.
## sudo apt install docker.io -y

Step 3
Start Docker service.
## sudo systemctl start docker
## sudo systemctl enable docker

Step 4
Add your user to Docker group.
## sudo usermod -aG docker $USER
## newgrp docker

Step 5
Verify Docker installation.
## docker --version
Check Docker is working.
## docker ps

output:
sakshi@LAPTOP-SJFPV9OV:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

2️⃣ Install kubectl
Step 6
Install kubectl.
## sudo snap install kubectl –classic
Verify installation.

## kubectl version --client

3️⃣ Install Minikube
Step 7
Download Minikube.

## curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-
    OUTPUT:
    sakshi@LAPTOP-SJFPV9OV:~$ curl -LO https://storage.googleapis.com/minikube/rele
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
```

```
100  128M  100  128M    0     0  1284k      0  0:01:42  0:01:42 --:--:--  680k


Step 8
Install Minikube.
## sudo install minikube-linux-amd64 /usr/local/bin/minikube

Step 9
Check version.
## minikube version

4️⃣ Start Kubernetes Cluster
Step 10
Start the cluster.
## minikube start

Step 11
Verify cluster status.
## kubectl get nodes
Expected output:
minikube Ready control-plane

    OUTPUT:
    sakshi@LAPTOP-SJFPV9OV:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE     VERSION
minikube   Ready    control-plane   6d21h   v1.35.1

5️⃣ Create Project Folder
Step 12
Create a working directory.
## mkdir ai-kubernetes
## cd ai-kubernetes

6️⃣ Create AI Application
Step 13
Create Python file.
## nano app.py
Paste:
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return "AI Model Running Successfully"

@app.route('/predict', methods=['POST'])
def predict():
    data = request.json['number']
    result = data * 2
    return jsonify({'result': result})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
Save:
CTRL + X
```

```
Y
Enter

7️⃣ Create Dockerfile
Step 14
## nano Dockerfile
Paste:
FROM python:3.10
WORKDIR /app
COPY . .
RUN pip install flask
EXPOSE 5000
CMD ["python","app.py"]
Save the file.

8️⃣ Build Docker Image
Step 15
Configure Docker to use Minikube.
## eval $(minikube docker-env)

Step 16
Build Docker image.
## docker build -t ai-model .

    wait till output show
    Successfully built 539040bc4024
Successfully tagged ai-model:latest


Verify image.
##docker images

9️⃣ Create Kubernetes Deployment
Step 17
Create deployment file.
## nano deployment.yaml
Paste:
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-model-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ai-model
  template:
    metadata:
      labels:
        app: ai-model
    spec:
      containers:
        - name: ai-container
          image: ai-model
          imagePullPolicy: Never
          ports:
```

```
            - containerPort: 5000
Save the file.

Step 18
Apply deployment.
## kubectl apply -f deployment.yaml
Verify pods.
## kubectl get pods

🔟 Create Service
Step 19
Create service file.
## nano service.yaml
Paste:
apiVersion: v1
kind: Service
metadata:
  name: ai-model-service
spec:
  type: NodePort
  selector:
    app: ai-model
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30007
Save.

Step 20
Apply service.
## kubectl apply -f service.yaml
Check services.
## kubectl get services

1️⃣1️⃣ Access Application
Step 21
Run:
## minikube service ai-model-service

Your browser will open and display:
AI Model Running Successfully

1️⃣2️⃣ Scale Pods (Important Step)
Step 22
Increase number of pods.
## kubectl scale deployment ai-model-deployment --replicas=5

Step 23
Verify scaling.
##kubectl get pods
You should see 5 pods running.

1️⃣3️⃣ Open Kubernetes Dashboard
Step 24
Launch dashboard UI.
## minikube dashboard
```

```
You can now see:
Pods
Deployments
Services
Nodes
```

assignment no 5

5. Create a Docker container for your AI model and test it on your local machine to ensure consistent performance across environments.

In [ ]:

```
folder:ai-model


vs code
## 3] Create app.py
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return "AI Model is Running!"

@app.route('/predict', methods=['POST'])
def predict():
    data = request.json['number']
    result = data * 2
    return jsonify({'result': result})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)

## 4] Create requirement.txt
add this info
flask
numpy

## 5] Create Dockerfile
add this info in Dockerfile

# Use official Python image
FROM python:3.9
# Set working directory
WORKDIR /app
# Copy files
COPY . /app
RUN pip install -r requirement.txt
COPY . .
# Expose port
EXPOSE 5000
# Run app
CMD ["python", "app.py"]
```

```
eg :-

6] Build Docker Image
Open terminal inside your project folder:
Run:
## docker build -t my-ai-app .

eg.

7] Run Docker Container
Run this:
## docker run -p 5000:5000 my-ai-app
eg .

8 ] Test It On Local Machine
Open browser and go to:
## http://localhost:5000
You should see:
AI Model is Running!
Eg .

9] Test Prediction API

## For Linux / Mac (or Git Bash)
curl -X POST http://localhost:5000/predict \
-H "Content-Type: application/json" \
-d '{"number": 5}'

## For Windows (PowerShell)
curl -X POST http://localhost:5000/predict `
-H "Content-Type: application/json" `
-d "{\"number\": 5}"


You should see: ( OUTPUT )

{"result":10}
```

In [1]:
```
## assi 9
```

In [ ]:
```
open vs code -vse1
create file add.py
git init
git add .
craete git repo -
git commit -m ""
git remote add origin https://github.com/sakshinikam2005/v10.git
git push -u origin main
sudo chown -R jenkins:jenkins /mnt/c/Users/Sakshi\ Nikam/OneDrive/Attachments/Deskt
***sudo chown file path ****
wsl
/mnt/c/Users/Sakshi\ Nikam/OneDrive/Attachments/Desktop/vsec10
*******
in jenkins
local host 8080
```

```
..new item
..frre sty
..none
..Poll SCM
?
in poll SCW
>>>
H/1 * * * *
>>>>>

In jenkins
envoronment
build step
excuted shell
    >>>>>
for this path go to terminal
pwd
copy this path
cd /mnt/c/Users/Sakshi Nikam/OneDrive/Attachments/Desktop/vsec10
   >>>> >>

Command
See the list of available environment variables
..cd /mnt/c/Users/Sakshi Nikam/OneDrive/Attachments/Desktop/vsec10
..python3 add.py
..git pull
..echo "build succesfully"
envvironment select from side
save
build now
console output
```

In [2]:
```
## ass 4
```

In [ ]:
```
Step-1 : give new item
Step-2 select freestyle project
Step-3 enter
Step-4 go to source code management
Step-5 select git
Step-6 paste git repo url
Strp-7 select Branch main
Step-8 select GitHub hook trigger
Step-9 add build step
Step-10 execute shell
Step-11 code write using echo
Step-12 ctrl s
Or Apply
Step-13 Back in new item in nav bar
Step-14 select build now
Green tya vr click
Select consol output


1]in git craete repo addtion
    aslo in this add add.py code
```

```
2]in unbtu terminal command
##sudo systemctl start jenkins.service

3]open jen localhost 8080
in jenkins
>> new item
>>freestyle project
>>enter
>>source code management
>>git
>> paste git repo ur(abvo created repo url)
>>in branch specifir
     */master instead do this */main

>>select GitHub hook trigger
>> build step
    >execute shell
    > write code
    echo "Build started"
    python3 add.py
    echo "Build finished"
>>save

after that  go github
>>go setting of repo
>>select webhooks
>> in that Add webhooks

again go ubantu terminal
type command
## ip addr
after see in ouput of terminal see same like
172.16.5.102/

go to githun in webhooks
>> select payload URL
type this https://172.16.5.102/Addition/
>> select content type
type this application/json


go to jenkins
>> select build now
```